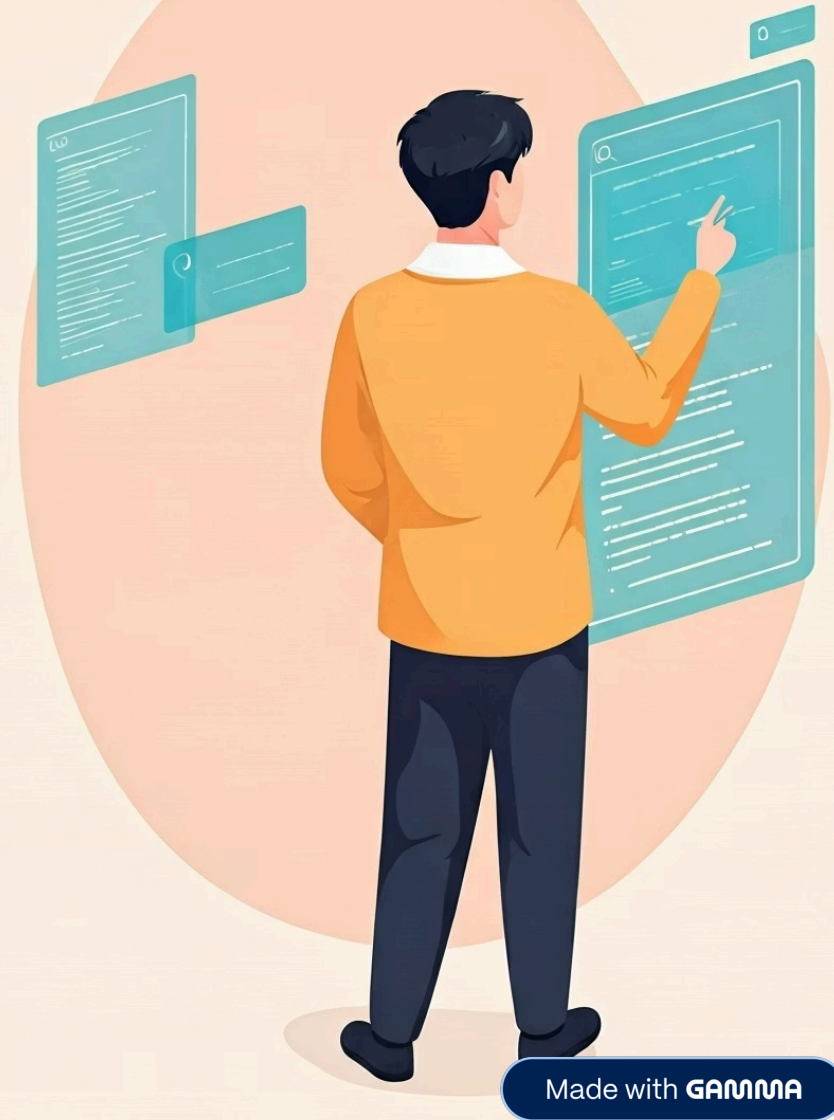


RAG 시스템 구축 가이드

검색 증강 생성으로 사내 코드베이스 이해하기

엔지니어와 데이터 과학자를 위한 실무 중심 접근법



RAG(검색 증강 생성)의 이해

기본 개념

LLM이 외부 지식 저장소(문서, 코드, DB 등)에서 관련 정보를 검색하여 답변 생성에 활용하는 방식입니다.

핵심 메커니즘

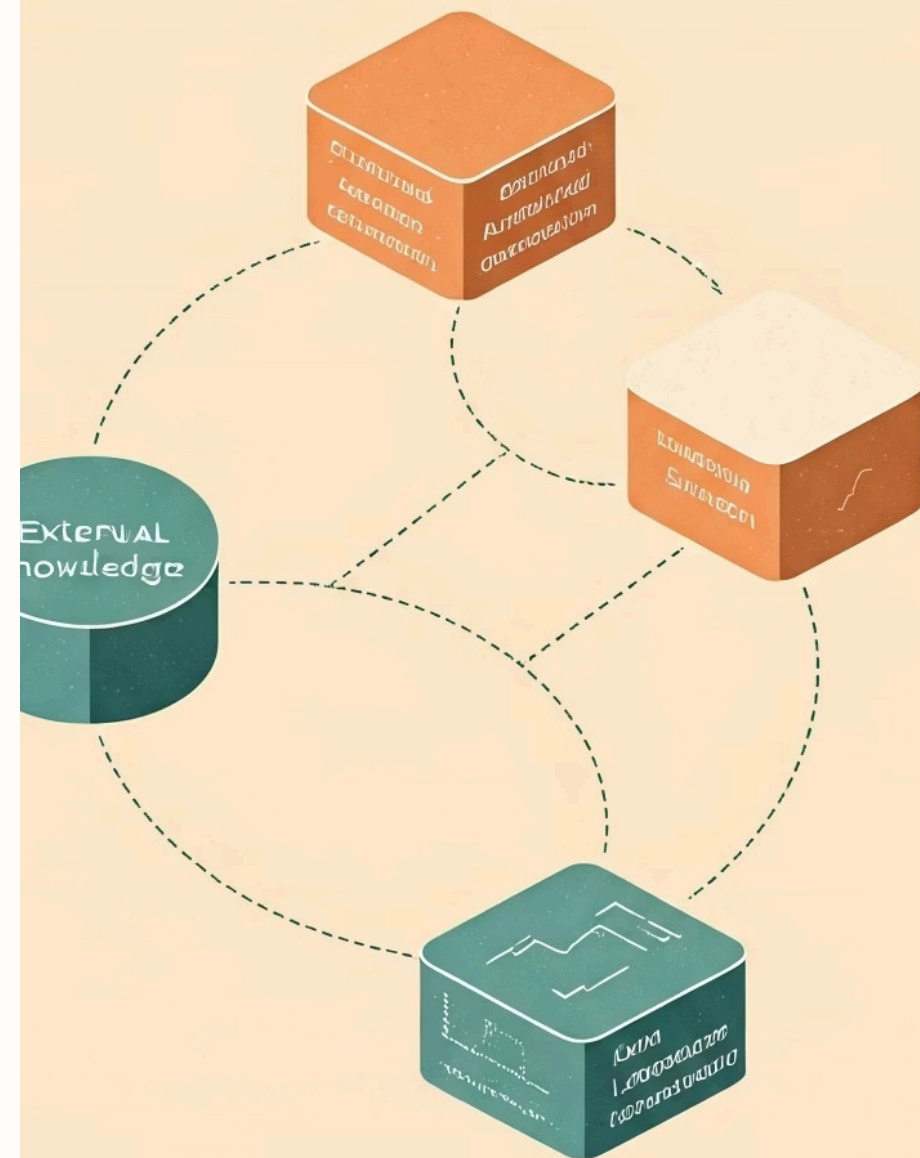
임베딩(벡터화) 기반 유사도 검색을 통해 관련 정보를 찾고, 이를 프롬프트에 삽입하여 LLM에게 컨텍스트를 제공합니다.

주요 장점

- 최신 정보 반영 가능
- 도메인 특화 지식 활용
- 환각(hallucination) 현상 감소

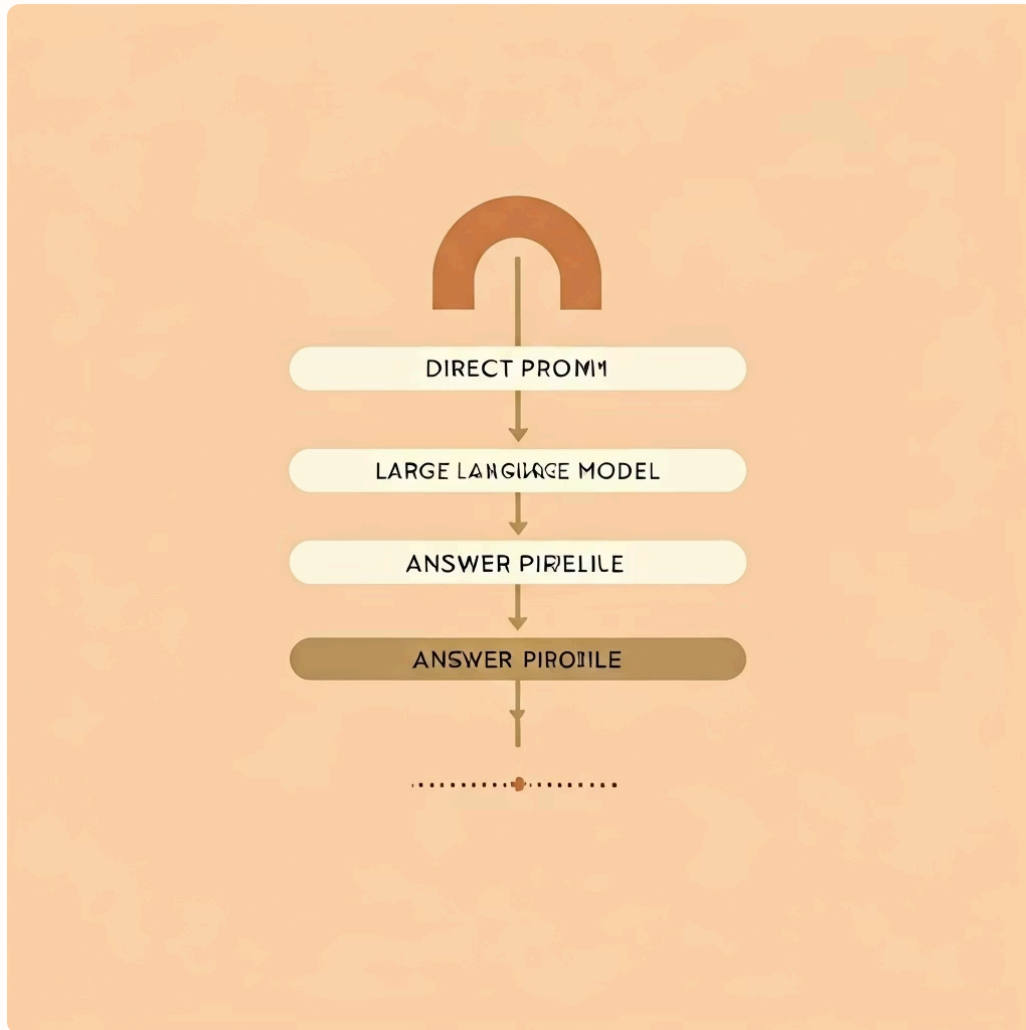
주의사항

데이터 갱신/정제가 필수적이며, 답변 생성 시 출처 표기를 통해 신뢰성을 확보해야 합니다.



비-RAG vs RAG 파이프라인 비교

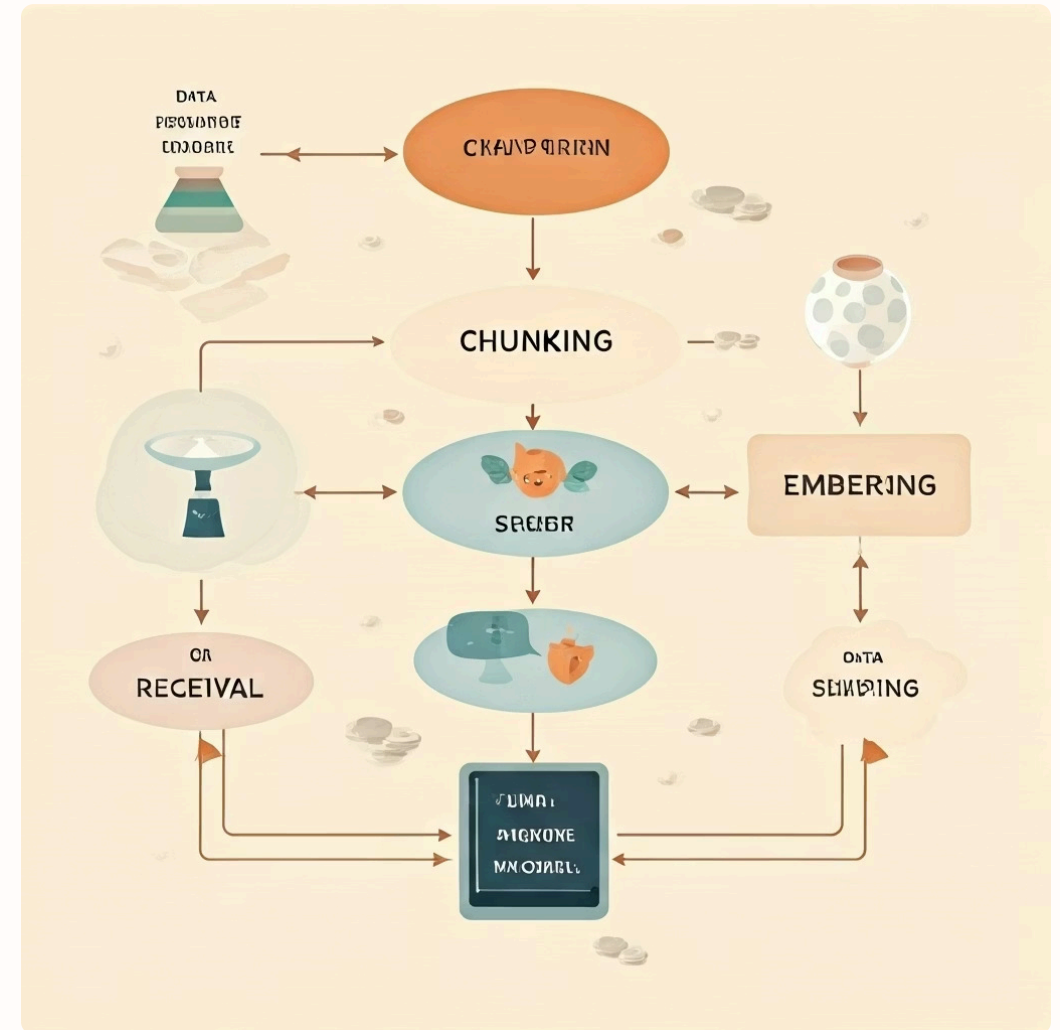
기존 방식 (비-RAG)



1. 사용자가 프롬프트 작성
2. LLM 직접 호출
3. 사전 학습 지식에만 의존한 답변 생성

한계: 최신 정보 부재, 특화 지식 부족, 환각 현상

RAG 방식



1. 데이터 수집·청크화 → 임베딩 → 인덱스 저장
2. 질문 임베딩 변환 → 벡터DB에서 Top-K 검색
3. 검색된 컨텍스트 + 질문을 LLM 프롬프트에 주입
4. 근거 기반 답변 생성

데이터셋 준비 원칙

데이터 품질 우선

양보다 질을 우선시하여 불필요하거나 중복된 데이터, 노이즈를 제거합니다.

최적 청크 크기

200~500 토큰 범위로 유지 (코드는 함수/클래스 단위, 문서는 문단 단위로 분리)

토큰이 너무 많으면 비용 증가 및 정밀도 하락

임베딩 모델 선택

텍스트 전용 모델만 사용하면 코드 유사도 인식에 한계가 있습니다.

코드 친화적 임베딩 모델 선택 또는 청크 헤더에 심볼/언어/경로 정보 포함

참고: 임베딩 모델과 생성 LLM은 독립적으로 운영 가능합니다. (같은 필요 없음)



프로토타입 아키텍처

기술 스택

- 언어/구조: HTML + JS + Java(Spring) + MyBatis XML
- 임베딩 모델: text-embedding-3-small
- 생성 모델: gpt-4o-mini (테스트용)
- 인덱스: FAISS(벡터), pkl/SQLite(메타데이터)

시스템 흐름도





테스트 결과: 공지사항 조회 예시

- 비-RAG

보편적인 구조만 제안 (사내 특화 코드 스타일 반영 못함)

범용적 MVC 패턴 설명에 그침

- RAG(Top-K)

MainController/Service/Mapper 패턴에 맞는 정확한 답변

사내 코딩 스타일 준수, 근거 표기 가능

- RAG+재랭킹

오히려 성능 저하 발생

데이터 규모가 작고 잘 정제된 경우 Top-K만으로 충분

결론: 소규모·정제 데이터셋에서는 Top-K 검색만으로도 최적의 결과를 얻을 수 있습니다. 재랭킹은 대규모·이질적 데이터셋에서만 효과적입니다.

주요 깨달음 및 교훈



데이터셋 품질

많은 데이터보다 정확한 데이터가 중요합니다. 기능별 타겟 데이터를 수집하고, 자연어 주석을 포함하세요.



임베딩 모델의 중요성

코드를 이해할 수 있는 임베딩 모델일수록 검색 성능이 향상됩니다. 도메인 특화 모델을 고려하세요.



목적별 LLM 라우팅

텍스트용과 코드용 모델을 분리 운영하면 각 도메인에 최적화된 응답을 얻을 수 있습니다.



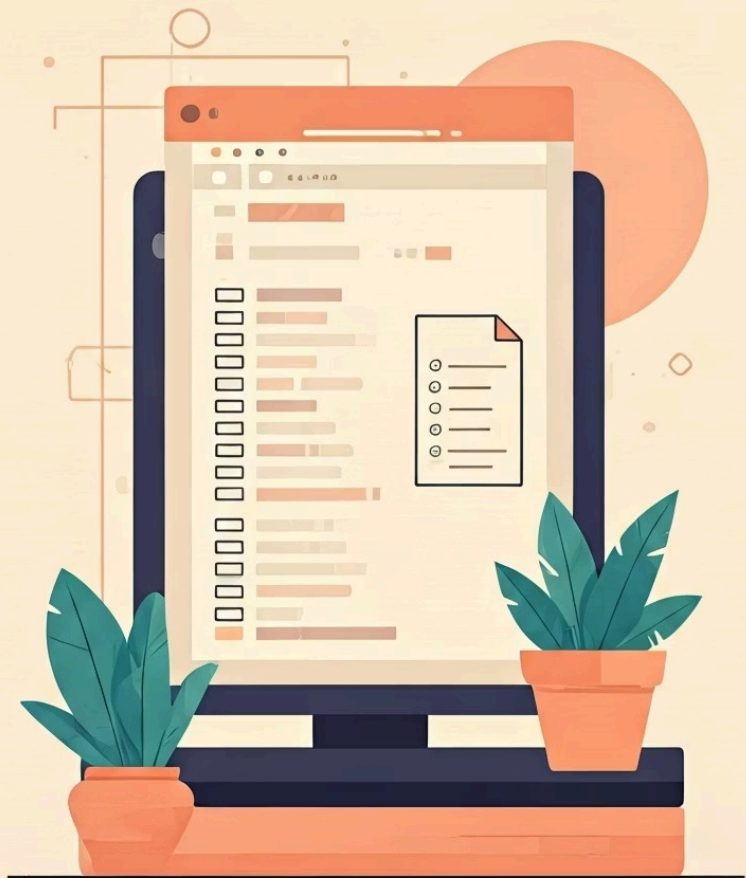
파인튜닝 활용

사내 용어/스타일 학습을 위해 Q&A 형식의 파인튜닝을 적용하세요. 지식 주입이 아닌 표현 일관성 강화에 집중합니다.



운영 관점

증분 업데이트(파일 해시 기반), 권한 필터 적용, 출처 표기 강제, 평가 지표 (R@K, Faithfulness, 지연 p95) 지속 모니터링이 필요합니다.



임베딩 파일 구성 및 주요 고민 포인트

임베딩 파일 구성

- 벡터 비교: FAISS 파일 (빠른 유사도 검색)
- 메타 관리: pkl 파일 (불필요 데이터 제거)
- Core 파일
(Controller/Service/SQL)은 반드시 포함 → 답변 일관성 확보

3가지 핵심 고민 포인트

1. Chunk 기준? (함수/클래스/SQL id 단위)
2. Chunk 데이터 쌓기? (중복 필터링 필요)
3. 메타데이터 타입 정의? (파일종류별 + 심볼/라인 범위 포함)

권장사항: Core 코드 위주로 수집하고
예외 케이스는 제외하여 일관성 있는 데이터셋을 구성하세요.

자주 묻는 질문(FAQ)

Q1. 임베딩 LLM이 꼭 필요한가?

네. 검색 증강은 임베딩 없이는 불가능합니다. 임베딩은 텍스트를 벡터 공간으로 변환하여 유사도 검색을 가능하게 하는 핵심 요소입니다.

Q2. FAISS+pk1만 있으면 OpenAI처럼 쓸 수 있나?

거의 가능합니다. 검색 + 프롬프트 조립 로직만 추가하면 OpenAI와 유사한 검색 증강 기능을 구현할 수 있습니다.

Q3. 재랭킹이 필수인가?

아닙니다. 상황에 따라 선택적으로 적용해야 합니다. 정제된 소규모 데이터셋에서는 오히려 불필요할 수 있습니다.

Q4. 임베딩 모델 CPU vs GPU?

둘 다 가능합니다. 대량 처리나 저지연이 필요한 경우에는 GPU를 추천합니다.

Q5. 벡터DB 업데이트 가능한가?

전용 벡터DB(Qdrant, Milvus 등)는 upsert를 지원합니다. 반면 FAISS 단독 사용 시에는 삭제/업데이트가 불편하여 재빌드를 권장합니다.

구현 체크리스트 및 결론

1

준비 단계

- Chunker (코드/문서 분리 규칙 적용)
- 임베딩 파이프라인 구축 (배치+증분)
- FAISS 인덱스 구축 (차원/정규화 일치)

2

검색 시스템

- 메타스토어 스키마 정의 (pkl/SQLite)
- 검색 API 구현 (Top-K, 권한 필터, 옵션: 재랭킹)
- 프롬프트 템플릿 설계 (근거 표기/금지 조항 포함)

3

운영 및 평가

- 모델 서빙 (Mistral-7B or gpt-4o-mini)
- OpenAI 호환 엔드포인트 래퍼
- 모니터링/로그/평가 스크립트

결론

RAG는 사내 코드베이스 활용에 강력한 도구입니다. 핵심 운영 원칙은 "양보다 질", "증분 업데이트", "출처 표기", "권한 필터"입니다. 작은 규모에서도 효과적인 RAG 시스템을 구축하여 개발 생산성을 높이세요.

